TECHNICAL REFERENCE

GG Program Documentation

Architecture, Code Structure & Module Reference

Green Garden — Cemetery Management System
Document version 1.0

Table of Contents

1. Overview & Technology Stack

Green Garden (GG) is a Django app built to run a memorial park's day-to-day operations: client and beneficiary records, lot and columbarium assignments, monthly payments, viewing and interment bookings on a slot-based calendar, employee accounts with role permissions, an activity log for accountability, and backups that run on their own.

What follows is a walkthrough of how the codebase is put together and how the pieces talk to each other.

Layer	Technology
Backend framework	Django 6.0.3 (Python)
Database	SQLite, accessed entirely through the Django ORM
Frontend	Django Templates + Bootstrap 5 + vanilla JavaScript
Reporting	openpyxl (server-generated .xlsx exports)
Authentication	Django's built-in auth system, extended with a staff profile model
Background tasks	Python threading (daemon threads) — no external task queue

> ***Note:*** *the code snippets in this document are trimmed down for clarity, not pasted in verbatim. Check the actual source files for the full implementation.*

2. Project Structure Reference

```
GG/ ← Django project folder
├── manage.py
├── GG/ ← project settings package
│ ├── settings.py ← configuration (INSTALLED_APPS, MIDDLEWARE, DB, etc.)
│ ├── urls.py ← top-level URL routing
│ └── wsgi.py / asgi.py ← deployment entry points
├── app/ ← the main Django application
│ ├── models.py ← database schema
│ ├── views.py ← request handling / page logic
│ ├── forms.py ← form definitions & field validation
│ ├── urls.py ← app-level URL routing
│ ├── admin.py ← Django admin model registrations
│ ├── middleware.py ← login rate limiting, admin area gate
│ ├── signals.py ← post_save / post_delete backup triggers
│ ├── backup.py ← ZIP backup builder, local + email delivery
│ ├── crypto.py ← encryption helper for stored settings
│ ├── apps.py ← app config; starts signals + daily scheduler
│ ├── context_processors.py← injects role flags into every template
│ ├── migrations/ ← schema history
│ ├── management/commands/ ← custom manage.py subcommands
│ ├── static/ ← source CSS / images
│ └── templates/app/ ← all HTML page templates
└── staticfiles/ ← collected static assets (build output)
```

3. Data Models

Everything that needs to persist lives in **app/models.py**. The schema stays pretty flat on purpose — mostly plain ForeignKeys instead of tangled many-to-many relationships — which keeps the queries throughout views.py easy to follow.

3.1 Client & Beneficiary

ClientPersonalInfo holds a client's personal details, spouse info, and government ID. There's a computed *full_name* property that just stitches together first, middle, and last name for display. **Beneficiary** sits underneath it as a child table (ForeignKey, related_name *beneficiaries*) — one row per dependent, with a name and relationship — managed inline on the same page rather than off in its own form.

Field group	Key Fields
Identity	client_first_name, client_middle_name, client_last_name
Contact / civil	client_address, client_contact_number, client_civil_status
Spouse (conditional)	client_spouse_name, client_spouse_date_birth, client_spouse_occupation, client_spouse_employer
Government ID	client_id_type, client_id_number, client_date_issued, client_place_issued

3.2 ClientStatus (Lot Plans)

This model represents a single lot or columbarium plan tied to a client. Over time a client can rack up several ClientStatus rows — a trail of cancelled and completed plans — but only one can be active and non-cancelled at any given moment.

What the fields actually mean depends on the *plan* field:

- **Standard lots** — Lawn lot, Garden lot, court tiers, estates — rely on phase, block, section, lot_number, and pa_number to pin down the physical location.
- **THS / THTC** plans reuse those same location fields, plus their own column_level field.
- **Columbarium** plans go a different route: columbarium_type, columbarium_level (1–4), and tomb_number — the standard location fields just stay null.

The financial fields — monthly_payment, duration, down_payment, discount_percent, balance, paid_balance — are what drive the Payment schedule that gets generated automatically. Rather than storing a separate discounted figure, a computed property applies the discount on the fly:

```
@property
def effective_down_payment(self):
if self.down_payment and self.discount_percent:
return self.down_payment * (1 - self.discount_percent / 100)
return self.down_payment
```

Cancelling a plan doesn't delete the row. is_cancelled, cancellation_reason, and cancellation_date just flip on instead, so the history sticks around for reporting.

3.3 Payment

Each Payment is one scheduled month tied to a ClientStatus (ForeignKey, related_name *payments*), kept in month order. It tracks the amount, whether it's paid, the date it was paid, and who processed it — processed_by gets set to null if that staff account is ever removed, so the payment history doesn't break.

3.4 Booking

A Booking is one Viewing or Interment appointment. booking_time only accepts 11 fixed hourly slots, 07:00 to 17:00, and that same choices list is shared with the calendar JavaScript on the front end so the two never drift apart. Double-booking a slot is blocked at the database level with a composite uniqueness constraint:

```
class Meta:
ordering = ['booking_date', 'booking_time']
unique_together = [('booking_date', 'booking_time')]
```

Status moves from Active to one of Completed, Cancelled, or No Show. Completed happens on its own (more on that in Section 9.2); Cancelled and No Show are something staff do manually, and both capture a reason and a timestamp.

3.5 UserLog (Employees)

UserLog is the employee profile, hung off Django's built-in User through a one-to-one link. It carries the role (General Staff or Financial Staff), personal details, an emergency contact, time_in/time_out tracking, a free-text field that just shows whatever the employee did most recently, and a hashed PIN used to confirm anything sensitive.

3.6 ActivityLog

ActivityLog is append-only — nothing in it ever gets edited or deleted. Every login, logout, and write action (adding or editing a client, plan, payment, booking, or employee) drops a new row with the staff member's name and role at that moment, what the action was, a short detail string, and a timestamp.

3.7 SystemSecret

SystemSecret is a generic key/value store for settings an admin needs to configure but that shouldn't be sitting in a source-controlled config file. Everything gets encrypted through app/crypto.py before it's saved, and only decrypted in memory when it's actually needed.

4. URL Routing & View Layer

app/urls.py wires each route to a function-based view over in app/views.py. The views are grouped by feature, and most of them follow the same shape: check permissions first, then handle the form or query, then render a template or return a JsonResponse. Anything that writes data usually calls a shared logging helper on its way out too.

Area	Representative Views	Notes
Auth	login_view, logout	Stamps session timestamps and writes a Login/Logout log entry
Clients	add_client_view, records_view, client_details_view, edit_details_view, delete_client_view	Deletion requires a confirmation step via a JSON request
Plans	plan, cancel_plan_view	Blocks assigning a new plan while an active one already exists
Payments	add_payment_view, payment_history_view	Doubles as the endpoint that marks a scheduled month as paid
Bookings	bookings_view, cancel_booking_view, calendar_view	Computes fully/partially booked day sets server-side
Lots	lots_view	Dynamic filtering depending on selected lot type
Employees	employee_view, add_employee_view, details_employee_view, edit_employee_view, delete_employee_view	Restricted to administrators
Monitor / System	monitor_view, backup_now_view, backup_status_view, system_settings_view	Administrative tooling
Exports	export_bookings_excel, export_payment_history_excel, export_lots_excel, export_records_excel	Each builds an openpyxl workbook and streams it back as a file response

Permission logic doesn't get repeated in every view — it's centralized in two reusable decorators:

```
def _admin_required(view_func):
... # redirects non-admins away with an error message

def _require_role(*allowed_roles):
def decorator(view_func):
... # admin always passes; staff must match an allowed role
return wrapper
return decorator
```

5. Forms & Validation Layer

Most of app/forms.py builds on a shared **BootstrapForm** base class. It tacks the form-control CSS class onto every field automatically and strips whitespace off submitted strings during clean(), so none of the individual forms have to repeat that boilerplate.

Form	Purpose

ClientForm / BeneficiaryFormSet	Client registration and editing, with an inline formset for beneficiaries
EmployeeCreateForm / EmployeeUpdateForm	Account and profile creation and editing for staff, including password and PIN handling
PlanForm	Lot or columbarium assignment, with a field set that adapts to the selected plan type
BookingForm	Viewing or interment booking, with a custom clean() that enforces the unique time-slot constraint

A handful of validator functions get reused across forms instead of copy-pasted:

- **clean_name()** — letters, spaces, and hyphens only, title-cased
- **clean_phone_number()** — takes either a 09XXXXXXXXX or 639XXXXXXXXX number and normalizes it down to one canonical 11-digit form
- **clean_date_of_birth()** — enforces a minimum age
- **clean_pin_field()** — requires exactly four digits and keeps any leading zeros intact

6. Template Layer Overview

Every page extends one shared **app/base.html** — top nav bar, collapsible sidebar, a restricted-access modal, and a small JS helper that disables buttons client-side before a request even goes out. That client-side check is just a courtesy, though; the real gatekeeping happens server-side in views.py.

Template	Backs
addclient.html / edit_details.html	Client create/edit, with a dynamic spouse section and beneficiary add/remove rows
plan.html	Plan assignment; toggles between Standard, THS/THTC, and Columbarium field groups
bookings.html / calendar.html	Slot-based scheduling UI with a mini calendar and a full month-view calendar
records.html / client-details.html / lots.html	Read, search, and listing views with confirmation-protected delete actions
payment_history.html / add_payment.html	Per-plan payment schedule and the in-page payment flow
viewemployee.html / add-employee.html / employee-edit.html	Employee record management
monitor.html / system_settings.html	Activity log, backup controls, and the settings form

7. Access Control & Middleware

app/context_processors.py works out the role flags once per request and hands them to every template automatically, so templates never have to go query the employee model themselves:

```
def role_context(request):
...
return {
'user_role': role,
'is_admin': False,
'is_general_staff': role == 'General Staff',
'is_financial_staff': role == 'Financial Staff',
}
```

That three-tier model — Admin, General Staff, Financial Staff — gets checked twice: once client-side for instant feedback, and again server-side inside each view, which is where the real security boundary lives.

Two custom middleware classes sit in settings.MIDDLEWARE and run on every single request:

Middleware	Responsibility
LoginRateLimitMiddleware	Tracks failed login attempts per client IP and blocks further attempts for a cooldown window once a threshold is exceeded
BlockNonAdminMiddleware	Redirects any request to the admin area away unless the user is authenticated and a superuser

8. Signals & Automated Backups

app/signals.py listens for post_save and post_delete on the core models, and kicks off a background backup any time something meaningful changes: a client added or edited, a plan change, a payment getting marked paid, a booking created or cancelled, an employee change.

```
@receiver(post_save, sender=Payment)
def backup_on_payment_save(sender, instance, created, **kwargs):
if instance.is_paid:
trigger_backup('payment_paid')
```

Triggering a backup spins up a daemon thread, so whatever request caused it doesn't sit around waiting. Inside app/backup.py, a lock combined with a cooldown check means a burst of rapid-fire triggers collapses down into one archive instead of ten. Each run bundles CSV exports of every major table plus a raw copy of the database into a single ZIP, then tries to deliver it locally and by email — independently, so one failing doesn't take the other down with it.

Separately, app/apps.py kicks off its own background thread when the app starts up. It sleeps until a configured time of day, fires a backup, and loops daily after that — with a guard in place so Django's autoreload during development doesn't end up starting two of these threads at once.

9. Key Business Logic

9.1 Plan Assignment & Payment Generation

Saving a plan triggers an internal helper that resets the ClientStatus fields, wipes out any Payment rows already generated for it, and recalculates the balance as monthly_payment times duration. From there, a second helper bulk-creates one Payment per month. There's no external date-math library involved — just a small pure-Python function that steps forward month by month:

```
def _add_months(dt, months):
total_months = dt.month - 1 + months
year = dt.year + total_months // 12
month = total_months % 12 + 1
return dt.replace(year=year, month=month, day=1)
```

Once the last scheduled month gets paid off, the plan automatically flips to inactive and gets stamped with a completion date — all inside the same atomic transaction as the balance update, so nothing gets left half-done.

9.2 Calendar Booking State Logic

For the requested month, the calendar view counts up active bookings per day and sorts each day into fully booked or partially booked, comparing against the fixed slot total — the template then uses that to color each cell. There's also a separate map of interment dates, built independently, but that's just there as a visual reminder and has no bearing on slot availability. Bookings flip from Active to Completed on their own once the date passes, so nothing needs a scheduled job to keep that status honest.

9.3 Confirmation Flow for Sensitive Actions

Anything destructive or financial — deleting a client or employee, cancelling a plan or booking, recording a payment — goes through one shared confirmation check tied back to the acting user's profile. On the front end, the confirmation modals always grab that value as a plain string rather than parsing it into a number, which matters because a PIN with a leading zero would otherwise get mangled.

10. Static Asset Organization

Rather than one big global stylesheet, CSS is split per page — addclient.css, booking.css, homepage.css, records.css, loginstyle.css — which makes it easy to find whatever rule you're hunting for. app/static/ is where things actually get edited; staticfiles/ is just the generated build output.

- Bootstrap 5 and its JS bundle get pulled from a CDN in the base and login templates.
- Page-specific behavior — calendar rendering, dynamic form sections, confirmation modals — lives inline in each template's script block rather than off in separate bundled files.
- There's print-specific styling too, which hides the nav chrome when printing client profiles or dashboards.

11. Code Conventions

- Views are function-based throughout, grouped by feature, with comment-banner headers inside views.py so you can jump around quickly.
- Write actions return a small JSON success/error payload rather than triggering a full page reload — that's what powers the in-page modals for delete, cancel, and pay.
- Choice lists — plan types, time slots, action types — live as class-level constants and get reused by both the model field and whatever form corresponds to it, so the two can't drift apart.
- Cancelling something never deletes the record. A status flag plus reason and date fields keep the full history around for reporting.
- A leading underscore on a function name marks it as an internal view-layer helper — nothing exposed via a URL.